

Topic 01: The "Works on My Machine" Curse: Environments

Basic Software Engineering Workflows

TOPIC 01

The "` Works on My Machine" Curse: Environments

Dependency isolation, venv/uv internals, sys.path resolution, and lockfiles.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Diagnose dependency collisions** in global Python `site-packages` and understand `sys.path` resolution.
- **Inspect virtual environment internals** (isolated binaries, symlinks, `pyenv.config`, and isolated packages).
- **Author modern packaging specifications** using declarative `pyproject.toml` (PEP 621 / PEP 517).
- **Enforce zero-drift reproducibility** with deterministic lockfiles (`uv.lock`) in production cloud pipelines.

The Dependency Collision Catastrophe

The Collision: Global Package Overwrites

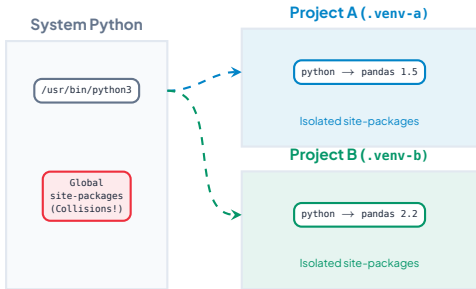
- **Legacy Model API (Project A):** Requires `pandas==1.5.3` and `numpy<1.24`.
- **New GenAI Pipeline (Project B):** Requires `pandas>=2.2.0` and `numpy>=1.26`.
- Running `pip install pandas==2.2` globally silently overwrites the files in `/usr/lib/python3.11/site-packages/`.
- **Production Outage:** Project A's production inference server crashes with `AttributeError`!

The "Aha!" Discovery: Path Sandboxing

- A Virtual Environment is not a VM or container; it is simply a **directory tree with a redirected `sys.prefix`!**
- CPython checks `pyvenv.cfg` on startup.
- `sys.path` prepends the local `.venv/lib/python/site-packages/` ahead of global directories.
- Multiple isolated environments coexist on the same machine without mutual interference.

Never execute `pip install`
PUNCHLINE outside an active virtual
environment.

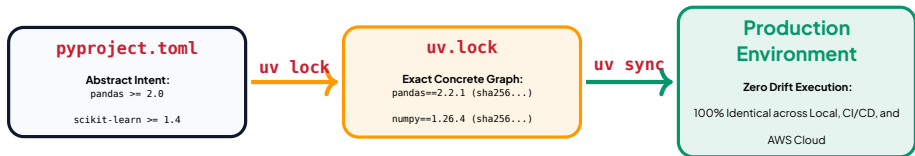
Hero Visualization: Virtual Environment Sandboxing



How CPython Resolves Imports

- **pyvenv.cfg:** Defines `home = /usr/bin` and `include-system-site-packages = false`.
- **sys.executable:** Points to the local environment binary `./venv/bin/python`.
- **sys.path Order:**
 1. Current Working Directory ("").
 2. Virtualenv **site-packages**.
 3. Python Standard Library.

Hero Visualization: Modern Packaging & Lockfile Pipeline



PUNCHLINE

Commit **uv.lock** to Git to guarantee bit-for-bit reproducible ML environments.

Production Checkpoint: High-Speed Operations with uv

Modern Cloud Python Toolchain

- Written in Rust, **uv** is 10 – 100× faster than standard **pip** and **virtualenv**.
- Global hardlink caching avoids copying duplicate wheels across projects.
- Replaces **pip**, **pip-tools**, **virtualenv**, **poetry**, and **pyenv**.

PUNCHLINE Sub-millisecond dependency synchronization in production.

Essential Production Commands

```
1 # 1. Create lightning-fast virtualenv
2 uv venv --python 3.11
3
4 # 2. Add dependencies with instant resolution
5 uv add pandas numpy scikit-learn
6
7 # 3. Deterministic sync from locked graph
8 uv sync --frozen
```